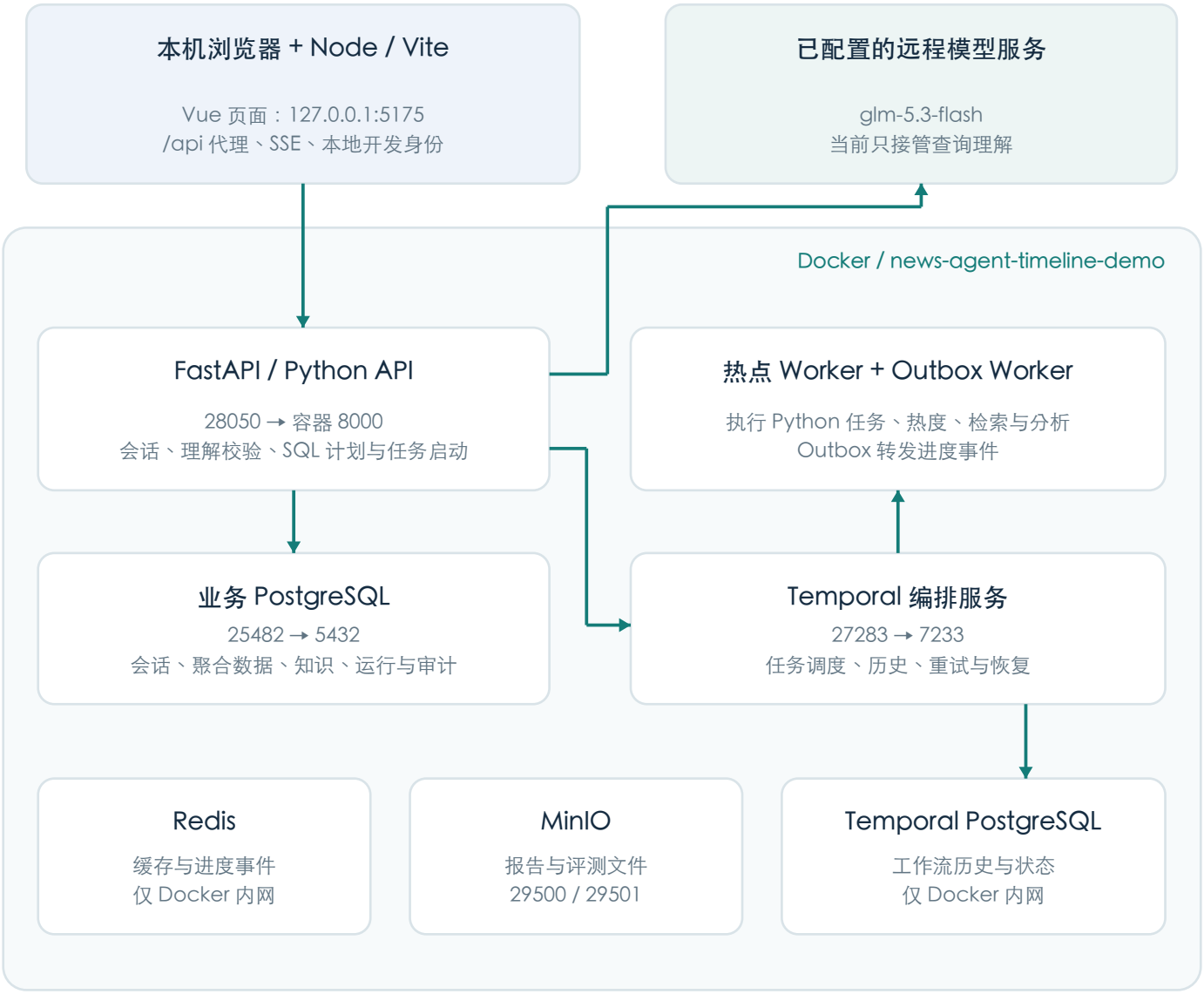
部署架构与启动手册

当前交互实例：5175 前端 + 28050 后端 | 本地演示环境

前端在你的电脑上运行；Python API、任务执行器和基础服务运行在 Docker 中。查询理解调用已配置的远程模型接口，热度与权威数字由 Python / SQL 计算。



API 和 Worker 都会读写业务 PostgreSQL，并按需要访问 Redis、MinIO 与知识索引。Temporal 安排任务，真正的项目 Python 代码由 Worker 执行。

直接入口： http://127.0.0.1:5175/chat
健康检查： http://127.0.0.1:28050/ready
本次核对： API healthy；PostgreSQL / Redis / Temporal 均为 ok。

阅读顺序：第2页看职责，第3页恢复现有环境，第4页了解重建，第5页看请求链路，第6页排障，第7页开发与生产，第8页查代码。

Docker 管什么，数据存哪里

容器状态与职责依据当前实例核对；配置中存在不等于当前已运行

服务	负责的工作	宿主机端口
api	FastAPI 接口、聊天、模型理解、Python 校验、SQL 快照和启动 Workflow	28050 → 8000
hot-news-worker	执行热点 Workflow / Activity；只读取数、指标与热度、正文和知识、分析与校验	无
outbox-worker	把业务数据库中的 Outbox 进度事件发送到 Redis	无
postgres	业务数据与事实源：会话、运行、聚合样本、知识与版本、SQL 审计	25482 → 5432
temporal	持久任务编排、重试和恢复；Python Activity 在热点 Worker 中执行	27283 → 7233
temporal-postgres	Temporal 自己的工作流历史与状态，与业务数据库分开	无
redis	缓存和事件推送；业务事实仍保存在 PostgreSQL	无
minio	对象存储：报告与评测 Artifact；另有管理控制台	29500 → 9000 29501 → 9001

一次性任务与可选模块

migrate 执行 Alembic 数据库迁移；**minio-init** 建桶并开启版本。本次均为 Exited (0)，表示任务成功完成，不需要常驻。

写作 Worker 与 Data Loop Worker 在 Compose 中有定义，但这次 timeline 实例未创建。RAG / 知识能力是 Python API 和 Worker 内的模块；新闻采集位于 news/，没有作为本次独立容器启动。

数据持久化

业务 PostgreSQL、Temporal PostgreSQL、MinIO 使用三个独立命名卷，分别保存业务状态、工作流状态和报告文件。Docker restart / start 会沿用它们。Redis 当前有匿名 /data 卷，但配置没有显式 AOF 策略，不能把它当作持久业务事实源。

API 与热点 Worker 使用已验收镜像 e58c617b4566…；Outbox 沿用较早镜像 b6c0917eae19…。前端是本机 Node 进程，远程模型由外部服务提供，两者均不在这些后端容器里。

现有环境：按顺序恢复

适用：现有容器和数据卷仍在；模型密钥已经注入原 API 容器

这条恢复路径沿用容器原有镜像与环境变量，不要求你重新输入模型密钥。步骤之间确认依赖健康后再继续。

1. 启动 Docker 与基础服务

```
docker desktop start --timeout 45
docker info --format '{{.ServerVersion}}'
docker start news-agent-timeline-demo-postgres-1 \
  news-agent-timeline-demo-temporal-postgres-1 \
  news-agent-timeline-demo-redis-1 \
  news-agent-timeline-demo-minio-1
```

在 Docker Desktop 中确认上述四个容器为 healthy；也可使用第6页的只读检查命令。

2. 启动 Temporal，再启动 API / Worker

```
docker start news-agent-timeline-demo-temporal-1
docker inspect --format '{{.State.Health.Status}}' \
  news-agent-timeline-demo-temporal-1
```

上一步输出 healthy 后，执行下面的业务启动命令：

```
docker start news-agent-timeline-demo-api-1 \
  news-agent-timeline-demo-hot-news-worker-1 \
  news-agent-timeline-demo-outbox-worker-1
curl -fsS --max-time 10 http://127.0.0.1:28050/ready
```

等待 /ready 返回 ready，postgres、redis、temporal 都为 ok。API 启动会完整核验约346万行小时指标和基线，可能需要数分钟；首次建库还会生成样本。

3. 在另一个终端启动前端

```
cd /Users/shi/Project/myProject/NewsAgent/agent/frontend
VITE_DEV_PROXY_TARGET=http://127.0.0.1:28050 \
VITE_DEV_PORT=5175 \
npm run dev -- --host 127.0.0.1 --port 5175 --strictPort
```

首次安装依赖或 package-lock.json 变化后先运行 npm ci；要求 Node.js ≥20.19.0。打开 http://127.0.0.1:5175/chat。保持前端终端运行，Ctrl-C 可停止。

仅重启业务进程时用 docker restart 后接现有 API / Worker 容器名。普通恢复不用运行迁移、建桶，也不用重启整个 Docker Desktop。

重建部署与三层 Compose

适用：创建容器、采用已批准镜像或改变启动配置；需要重新提供安全环境注入

文件	作用
docker-compose.native-e2e.yml	基础服务、原生 Python API / Worker、本地 Port 与一次性任务
docker-compose.timeline-demo.yml	项目名、timeline-v4 数据集、28050 等端口和 timeline-demo 镜像
docker-compose.timeline-query-model.yml	仅给 API 挂载已有模型连接文件，并注入查询理解凭证

以下命令以已有的 timeline-demo 验收镜像为前提。当前终端须通过原有安全机制注入 NEWSAGENT_MODEL_API_KEY；已有连接文件 model-runtime.compatible.yml 只读挂载。不把真实密钥写进命令、PDF 或前端配置。

创建部署：自动按依赖启动、迁移和建桶

```
cd /Users/shi/Project/myProject/NewsAgent/agent
docker compose --env-file /dev/null \
  -f deploy/docker-compose.native-e2e.yml \
  -f deploy/docker-compose.timeline-demo.yml \
  -f deploy/docker-compose.timeline-query-model.yml \
  up -d --no-build --pull never \
  api hot-news-worker outbox-worker
```

新数据卷会触发初始化；已有数据库的迁移版本必须与所选镜像兼容。缺少镜像、凭证或连接文件时先补齐对应配置，不能把远程理解静默降级为规则模拟。

基础服务已健康：只更新 API 和热点 Worker

```
docker compose --env-file /dev/null \
  -f deploy/docker-compose.native-e2e.yml \
  -f deploy/docker-compose.timeline-demo.yml \
  -f deploy/docker-compose.timeline-query-model.yml \
  up -d --no-deps --no-build api hot-news-worker
```

up 会采用目标镜像并可能重新创建容器，因此要重新注入所需环境变量。--env-file /dev/null 避免 Compose 自动加载 .env。restart 只重启原容器，不会应用新源码、新镜像或新环境配置。

命令只说明部署方式，本次制作 PDF 没有执行重建、升级数据库或调整生产发布。

一句话怎样变成查询与报告

模型提出结构化意图；Python 审批执行条件；SQL 与热点运行使用同一窗口

1 前端入口

/chat → /api/v1/conversations/{id}/messages/stream；Vite 代理注入本地身份

↓

2 API 保留原问并理解

ConversationAgentService → query_hot_news → _understand；一次服务器时钟、原问筛查、v4 模型 Prompt

↓

3 Python 独立校验

extract_query_time + resolve_query_understanding；核对已识别的时间、数量、指标、筛选、权限和数据覆盖

↓

4 固定 SQL 编译与绑定

SqlAssistantService.preview → compile_query → AST Guard / 参数绑定 / 租户限制；形成审计快照和同窗口 run

↓

5 Temporal / 热点 Worker

PostgreSQL 只读候选与基线 → Python 权威热度 → 正文 / 知识检索 → 分析 Port 与 Validator

↓

6 保存与返回

PostgreSQL / MinIO 保存运行和报告；Redis 进度 → API SSE → 会话结果 → Vue 展示

当前真实模型范围：查询理解使用 glm-5.3-flash + native-query-understanding-v4。其他本地演示模型环节仍是隔离规则 Port；没有第二次让模型编写可执行 SQL。

当前数据范围：2026-09-10 00:00 至 2026-10-10 00:00（北京时间，结束不含），每次一个完整小时。补全年份使用服务端当前年，不按样本日期偷偷改写。

试用：10月9号早上八点到九点热度最高的前五条新闻，帮我总结一下。模型结构错误、调用失败或已识别条件漂移时停止执行；陌生业务表达仍需质量评测。

健康检查、排障与停止

先区分前端、代理、API、依赖、Worker，再区分理解错误与数据范围限制

三个最直接的检查

```
docker ps --filter name=news-agent-timeline-demo
curl -fsS --max-time 10 http://127.0.0.1:28050/ready
curl -fsS --max-time 10 \
  http://127.0.0.1:5175/api/v1/conversations/runtime
```

/runtime 经前端代理访问，可同时验证代理身份与 API。查看 query_scope.understanding_model：provider=openai_compatible、model_route=glm-5.3-flash、prompt_version=native-query-understanding-v4。

现象	检查重点
5175 无法打开 / 端口占用	确认 npm run dev 在运行；只停止占用5175的本项目进程。不要批量结束所有 Node。
页面能打开，API 502 / 连接失败	检查28050的 /ready；代理目标需为28050，当前实例不要混用5174 / 28000。
401 / 403	通过5175代理访问；本地身份配置由 dev-server/local.yml 在 Node 中读取。不要把模型密钥填入浏览器。
启动时 API health: starting	数仓完整 hash 校验可能较慢；检查CPU和启动状态，等待 /ready，不跳过完整性校验。
任务提交后没有完成	除 /ready 外，还要确认热点 Worker 正在运行；健康接口不等于 Worker 已完成任务验收。
自然语言被澄清或拒绝	区分歧义、缺数据、不支持条件、模型结构错误、攻击；当前“今日”不在样本范围。

只读查看单个容器状态

```
docker inspect \
  --format '{{.State.Status}} {{.State.Health.Status}}' \
  news-agent-timeline-demo-api-1
```

停止时保留恢复条件

前端按 Ctrl-C。后端可 docker stop 现有业务容器，再停止依赖，容器、原环境和数据卷保留。down 会删除容器和网络，下次须重新注入模型凭证；down -v 还会删除数据卷，不用于日常重启。

只有 Docker 引擎本身无响应才考虑 docker desktop restart --timeout 45。它影响整个 Docker Desktop 的所有本地容器。正常停止失败后才考虑官方强制停止与重新启动；不要执行工厂重置或删除卷来解决普通启动问题。

改代码与生产部署的区别

当前是本地隔离演示；代码开发、镜像构建、配置发布和生产验收是不同步骤

后端：源码进入镜像，重启不等于更新

Dockerfile.e2e 将 app、examples、迁移、公共配置等 COPY 到 /app。当前 API 没有挂载工作区源码，也没有启用 uvicorn --reload。因此你修改本机 Python 文件后，原容器不会自动加载。

采用新代码应先审查工作区改动、运行相应测试、构建候选镜像，再进行版本兼容检查和部署。下面只是构建示例，不会自动激活候选，也不能代表当前工作区全部通过验收。

```
cd /Users/shi/Project/myProject/NewsAgent/agent
docker build -f Dockerfile.e2e \
  -t news-agent/writing-agent-service:timeline-candidate .
```

要采用候选，应显式更新已批准部署配置中的镜像引用；不要以为构建 timeline-candidate 后重启仍引用 timeline-demo 的容器就会生效。数据库迁移和镜像必须配套。

前端：开发直接读源码，生产交付静态文件

Vite 开发服务读取当前 Vue / TypeScript 源码并支持热更新；更改启动 YAML 或代理变量后重启。npm run build 会先类型检查，再生成 frontend/dist 静态资源，本身不会启动 Python 后端。

生产部署应采用的边界

层	生产方式与尚需验收的部分
浏览器与网关	静态资源由Web服务器/CDN提供；可信Gateway注入身份，使用TLS。开发身份代理不用于生产。
API 与 Worker	使用经过测试的版本化镜像、批准Prompt/模型绑定；独立伸缩与日志审计。
数据与存储	批准企业聚合接口、只读查询权限、持久卷/备份、保留策略和网络隔离需验证。
模型与知识	企业推理/Embedding合同测试、配额、结构化输出与向量索引能力需联调。

当前运行的是验收镜像快照，并不包含工作区所有正在变化的文件。优先恢复5175交互时沿用现有容器；生产 Prompt、规则、模型与发布变更继续版本化并人工审批。

TODO：真实问法质量评测与模型结构稳定性、未知业务条件语义层、实时水位、跨小时去重/全天查询、企业Adapter、生产负载与故障恢复。当前没有把 StarRocks 物化视图作为部署依赖。

代码、配置与验证入口

源码根目录已为 agent/；镜像名称 writing-agent-service 是保留的历史命名

部署三层文件（基础 / 数据集 / 查询模型）

/Users/shi/Project/myProject/NewsAgent/agent/deploy/docker-compose.native-e2e.yml

/Users/shi/Project/myProject/NewsAgent/agent/deploy/docker-compose.timeline-demo.yml
/Users/shi/Project/myProject/NewsAgent/agent/deploy/docker-compose.timeline-query-model.yml

前端代理与配置加载：vite.config → loadLocalDevSettings

/Users/shi/Project/myProject/NewsAgent/agent/frontend/vite.config.ts

/Users/shi/Project/myProject/NewsAgent/agent/frontend/dev-server/local-config.mjs
/Users/shi/Project/myProject/NewsAgent/agent/frontend/dev-server/local.yml（已有本地配置，本文不展示其值）

API 启动：create_local_app / local_lifespan → build_local_hot_news_sql_service

/Users/shi/Project/myProject/NewsAgent/agent/examples/local_simulation.py

会话入口：ConversationAgentService → query_hot_news → _understand

/Users/shi/Project/myProject/NewsAgent/agent/examples/conversation_hot_news.py

模型连接投影与独立客户端；时间解析；意图复核

/Users/shi/Project/myProject/NewsAgent/agent/app/model_runtime/query_understanding.py

/Users/shi/Project/myProject/NewsAgent/agent/app/conversation/query_time.py
/Users/shi/Project/myProject/NewsAgent/agent/app/conversation/query_understanding.py

SQL 核心：preview → compile_query → Guard → 只读 PostgreSQL

/Users/shi/Project/myProject/NewsAgent/agent/app/sql_assistant/service.py

/Users/shi/Project/myProject/NewsAgent/agent/app/sql_assistant/planner.py
/Users/shi/Project/myProject/NewsAgent/agent/app/sql_assistant/guard.py
/Users/shi/Project/myProject/NewsAgent/agent/app/sql_assistant/warehouse.py

热点 Worker 与 SQL 数据/知识适配

/Users/shi/Project/myProject/NewsAgent/agent/examples/native_hot_news_e2e_worker.py

/Users/shi/Project/myProject/NewsAgent/agent/examples/native_hot_news_sql_support.py

配置项、测试与证据

关键配置：VITE_DEV_PROXY_TARGET / VITE_DEV_PORT；MODEL_RUNTIME_CONFIG_PATH；CONVERSATION_QUERY_UNDERSTANDING_CONFIG_PATH；SQL_ASSISTANT_DATASET_PROFILE / SCENARIOS_PATH；DATABASE_URL、TEMPORAL_ADDRESS、REDIS_URL、ARTIFACT_ENDPOINT。凭证由安全环境机制注入。

后端定向测试（同目录另有 runtime / integration / guard 模块）

/Users/shi/Project/myProject/NewsAgent/agent/tests/test_query_time.py

后端离线：在 agent/ 的依赖环境中执行 python -m pytest -q -m 'not e2e and not integration and not live'。前端：在 agent/frontend/ 执行 npm test、npm run build；真实模型质量评测需单独进行。

详细调用链、此前测试与真实模型验收记录

/Users/shi/Project/myProject/NewsAgent/agent/docs/query-understanding-natural-time.md

本次文档核对了容器元信息、端口、持久卷、/ready 与前端 /runtime；没有重新运行全部测试。采集 → Python 切片/Embedding/索引 → 检索 → 分析/报告的知识链路由本项目实现。